

TRINNITY VISERON SYSTEM — ANÁLISIS DE PATENTES

¿Qué se puede patentar y por qué?

© Pedro Costa (Comandante) · Trinnity Hurtado (Rainha)

RESUMEN EJECUTIVO

El TVS contiene **5 invenciones patentables** que resuelven problemas reales en el mercado de IA autónoma. Cada una tiene **novedad técnica**, **aplicación industrial** y **ventaja competitiva** clara. El costo estimado de registro es €5.000-15.000 por patente (Europa/España) y toma 12-24 meses.

INVENCIONES PATENTABLES

1. VAEC — Autonomous Evolution & Continuity Policy

¿Qué es? Un sistema de políticas de evolución autónoma con gates obligatorios que impiden que cualquier cambio se promueva sin pasar por verificación completa.

¿Por qué es patentable?

- **Novedad:** No existe ningún sistema de IA que强制amente requiera gates IMPLEMENT → TEST → SYNC → BUILD → VERIFY → LEARN → PROMOTE antes de promover cambios
- **Problema resuelto:** Los sistemas de IA autónoma pueden hacer cambios destructivos; VAEC garantiza que NINGÚN cambio se promueva sin verificación
- **Aplicación industrial:** Aplicable a cualquier sistema de IA que opere autónomamente (empresas, fábricas, hospitales, trading)
- **Ventaja:** Rollback automático si algún gate falla; journal completo para auditoría

Código fuente: `src/omega/evolution/VaecOrchestrator.ts` **Claims sugeridos:**

- Método de evolución autónoma con gates secuenciales obligatorios
- Sistema de rollback automático basado en estado persistente
- Journal de evolución para auditoría regulatoria

2. OMEGA Kernel — Event-Driven Task Execution with Verification

¿Qué es? Un kernel de ejecución de tareas con pipeline completo: plan → fila → agente → herramientas → ejecución → **verificación** → resultado, con persistencia y recuperación automática.

¿Por qué es patentable?

- **Novedad:** La verificación automática post-ejecución (TaskVerifier) con reglas configurables no existe en otros frameworks de agentes
- **Problema resuelto:** Los agentes ejecutan pero no verifican; OMEGA verifica cada tarea contra reglas de schema, evidencia, invariantes y política
- **Aplicación industrial:** Aplicable a cualquier sistema que requiera ejecución autónoma verificable (finanzas, salud, industria 4.0)

- **Ventaja:** `Verified Task Completion Rate` como métrica de confianza; persistencia de tareas pendientes en restart

Código fuente: `src/omega/kernel/TaskQueue.ts`,
 `src/omega/verifier/TaskVerifier.ts` **Claims sugeridos:**

- Pipeline de ejecución de tareas con verificación post-ejecución automática
- Sistema de reglas de verificación componibles (composite verifier)
- Recuperación automática de tareas pendientes tras reinicio del sistema

3. AutonomyOS — Domain-Specific Autonomy Levels with Policies

¿Qué es? Un sistema de niveles de autonomía por dominio (finanzas, deploy, datos, mensajería, agentes, investigación, sistema) con políticas configurables que determinan qué puede hacer la IA sin supervisión humana.

¿Por qué es patentable?

- **Novedad:** Niveles de autonomía 0-5 por dominio con políticas `autoBelow`, `approvalFrom`, `denyAbove`, `requireApprovalFor`, `denyFor` no existen en otros sistemas
- **Problema resuelto:** La IA autónoma necesita límites por dominio (no es lo mismo operar finanzas que investigar)
- **Aplicación industrial:** Aplicable a cualquier organización que use IA con diferentes niveles de riesgo por área
- **Ventaja:** Auditoría completa de decisiones; cada operación tiene veredicto registrado

Código fuente: `src/omega/autonomy/AutonomyOS.ts` **Claims sugeridos:**

- Sistema de niveles de autonomía configurables por dominio
- Motor de veredictos con políticas de aprobación multi-nivel

- Registro de auditoría de decisiones autónomas

4. Squad Formation with Dual Leaders (Pedro/Trinnity Model)

¿Qué es? Un sistema de formación de equipos de agentes (squads) con dos líderes principales (estratégico + técnico) que tienen permisos diferenciados y supervisan operaciones conjuntas.

¿Por qué es patentable?

- **Novedad:** El modelo de liderazgo dual donde un líder estratégico (CEO) y uno técnico (Architect) co-gobiernan squads con permisos diferenciados no existe en frameworks de agentes
- **Problema resuelto:** Los sistemas de agentes carecen de jerarquía de mando clara; el TVS establece cadena de mando con Pedro (estrategia) y Trinnity (técnica)
- **Aplicación industrial:** Aplicable a organizaciones autónomas donde la IA necesita supervisión humana en capa estratégica
- **Ventaja:** Escalable a N líderes por dominio; cada squad tiene permisos granulares

Código fuente: ``src/core/squads/SquadManager.ts``,
``src/omega/squads/SquadRegistry.ts`` **Claims sugeridos:**

- Sistema de formación de equipos con líderes duales de roles diferenciados
- Registro de permisos por squad con herencia de líderes
- Ejecución de squads con resolución de miembros en runtime

5. Skill Execution Fabric (P0.2)

¿Qué es? Un sistema que conecta habilidades declaradas (skills) con ejecución real en tiempo descubrimiento, validación de contratos y persistencia de experiencia.

¿Por qué es patentable?

- **Novedad:** La combinación de SkillExecutor + SkillContractRegistry + ExperienceStore para ejecución autónoma verificable de habilidades no existe
- **Problema resuelto:** Los frameworks de IA tienen skills pero no verifican si se ejecutaron correctamente ni aprenden de la experiencia
- **Aplicación industrial:** Aplicable a cualquier sistema que necesite aprender y ejecutar habilidades nuevas autónomamente
- **Ventaja:** Las skills se descubren, validan, ejecutan y registran en ciclo cerrado

Código fuente: ``src/core/intelligence/SkillExecutor.ts``,

``src/core/intelligence/SkillContractRegistry.ts``,

``src/core/memory/ExperienceStore.ts`` **Claims sugeridos:**

- Sistema de ejecución de habilidades con validación de contratos
- Almacén de experiencia para aprendizaje de ejecuciones pasadas
- Pipeline de descubrimiento → validación → ejecución → registro

INVENCIONES ADICIONALES (potencialmente patentables)

6. EventBridge con Wildcards y Filtrado por Fuente

- **Innovación:** Sistema de eventos con wildcards (``task.*``, ``memory.*``), filtrado por fuente y ring buffer de historial
- **Código:** ``src/omega/kernel/EventBus.ts``, ``src/omega/kernel/EventBridge.ts``

7. KnowledgeGraph con Community Detection

- **Innovación:** Grafo de conocimiento con detección automática de comunidades, god nodes y conexiones sorprendentes
- **Código:** `src/core/memory/KnowledgeGraph.ts`

8. Governance Bíblica como Capa de Seguridad

- **Innovación:** Sistema de gobernanza basado en principios éticos que bloquea operaciones fraudulentas
- **Código:** `src/core/governance/bible.ts`

PROCESO DE PATENTAMIENTO

Pasos a seguir:

- **Buscar anterioridad** (€500-1.000): Verificar que no exista nada igual en patentes registradas
- **Redactar solicitud** (€3.000-8.000 por patente): Descripción técnica + claims + dibujos
- **Presentar en SPTO** (España) o EPO (Europa): €1.000-3.000
- **Examen técnico** (12-24 meses): El examinador verifica novedad y actividad inventiva
- **Concesión** (€500-1.000): Se concede la patente por 20 años

Costo total estimado:

- **España (SPTO):** €5.000-8.000 por patente
- **Europa (EPO):** €10.000-15.000 por patente
- **Internacional (PCT):** €20.000-30.000 por patente

Recomendación:

Empezar con **2 patentes prioritarias:**

- **VAEC** (más novedoso, más fácil de defender)
- **OMEGA Kernel** (más aplicable industrialmente)

Costo total: €10.000-20.000 para protección en España + Europa.

DOCUMENTOS GENERADOS

Documento	Descripción
`Viseron_Relatorio_Estado.pdf`	Estado actual del sistema + lo que funciona
`Viseron_Update_Report_2026-08-25.pdf`	Últimos cambios y commits
`Viseron_Plano_Estrategico_2026-08-25.pdf`	Estrategia a 90 días

`Viseron_Roadmap_Tecnico_2026-08-25.pdf`	Roadmap técnico detallado
`Viseron_100_Melhorias_Integracao.pdf`	100 mejoras integradas
`Viseron_Pipeline_Receita.pdf`	Pipeline de cobranzas reais
`Viseron_Analisis_Patentes.pdf`	Este documento

Comandante: Pedro Costa **Rainha:** Trinnity Hurtado **Fecha:** 25 de Agosto de 2026 **Sistema:** Trinnity Viseron System v7.0.0

© **Pedro Costa (Comandante)** · **Trinnity Hurtado (Rainha)**

Trinnity Viseron System v7.0.0 — 2026-08-25